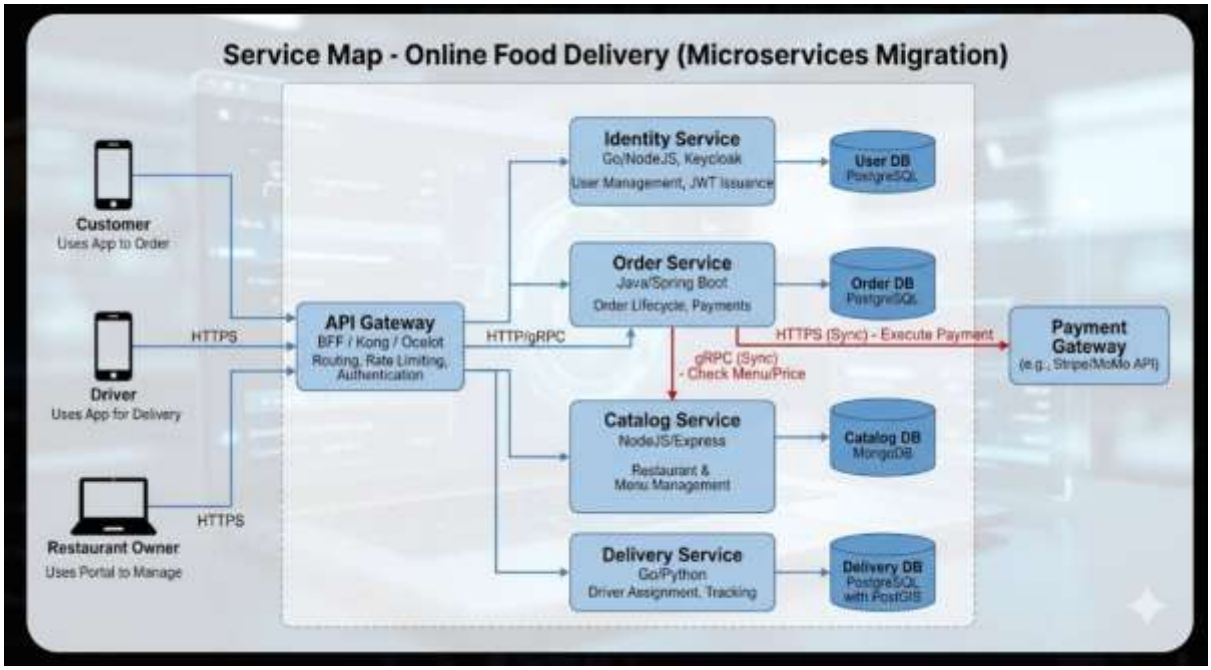
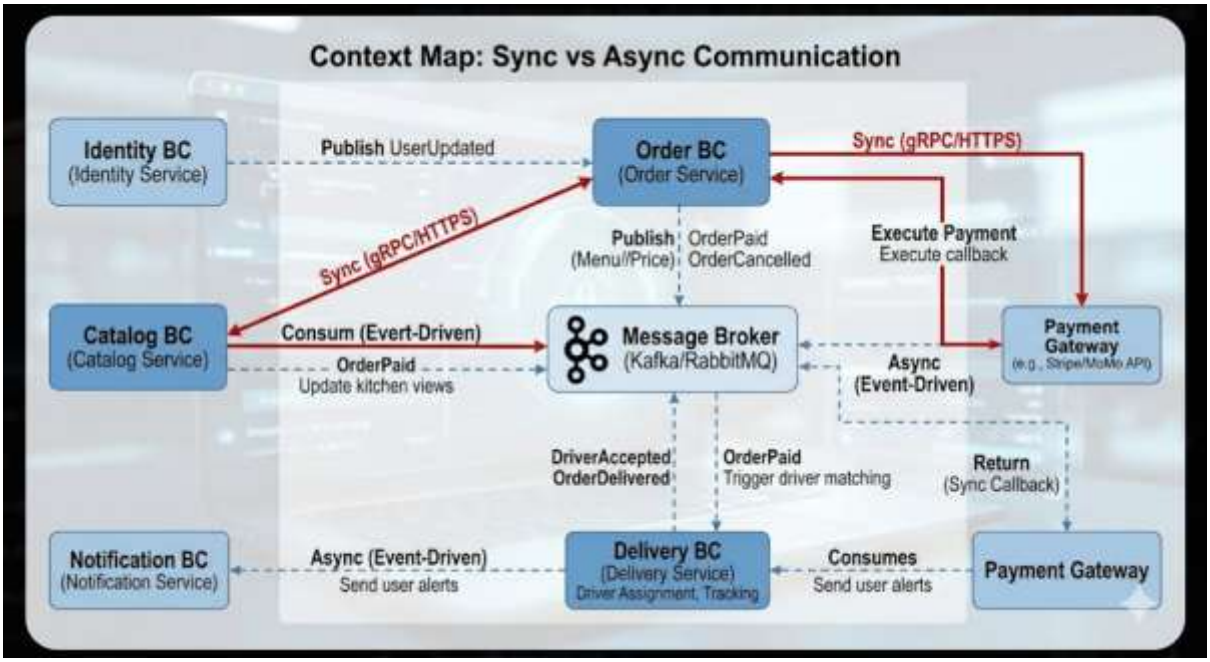
BÀI TẬP: MONOLITH TO MICROSERVICES MIGRATION

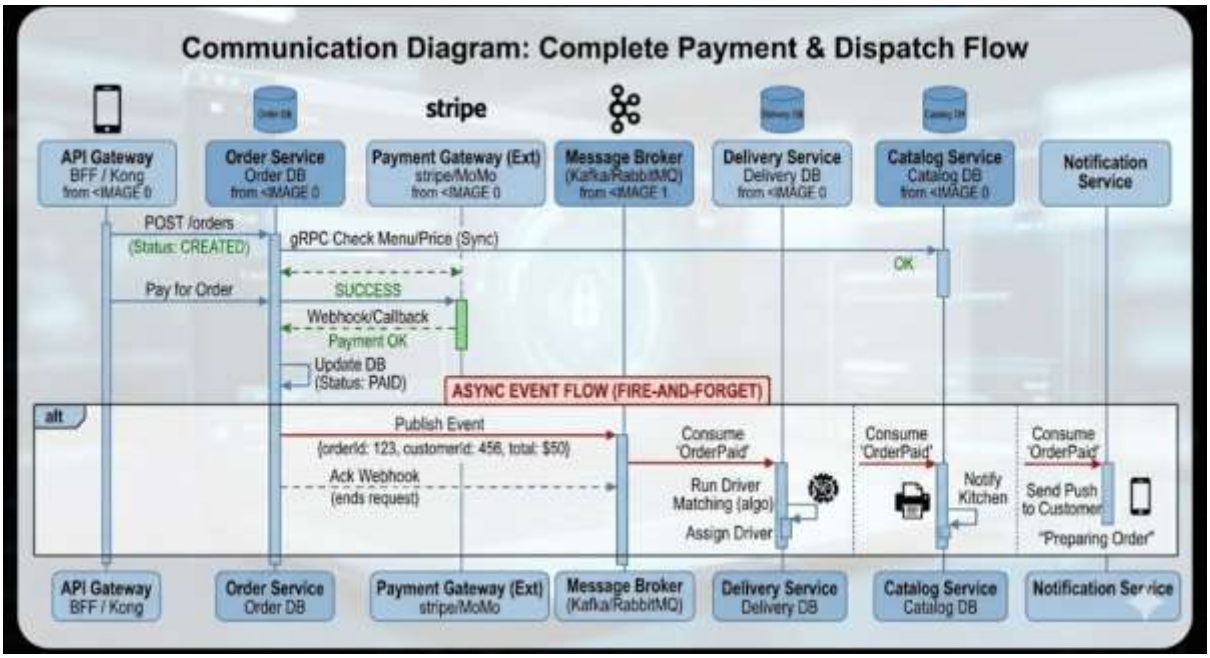
Hệ thống: Online Food Delivery Phạm vi: 3 Chức năng cốt lõi:

1. Quản lý Khách hàng & Đặt hàng (Order Management): User tìm món, tạo đơn, thanh toán.
2. Quản lý Nhà hàng & Thực đơn (Restaurant & Menu Management): Nhà hàng quản lý món ăn, xác nhận đơn.
3. Quản lý Giao hàng (Delivery Management): Tài xế nhận đơn, cập nhật trạng thái giao hàng.

1.Thiết kế bản monolith → microservices migration.







2. Xác định bounded context

Trước khi tách service, chúng ta cần xác định các ranh giới nghiệp vụ (Bounded Context) rõ ràng để đảm bảo tính Loose Coupling (ít phụ thuộc).

Dựa trên 3 chức năng yêu cầu, ta xác định được 3 Bounded Contexts chính như sau:

Bounded Context	Trách nhiệm chính (Responsibility)	Các Entity/Aggregate chính	"Ngôn ngữ chung" (Ubiquitous Language) nổi bật
Identity & Customer Context	Quản lý thông tin người dùng (Khách, Chủ quán, Tài xế), xác thực, phân quyền.	User, Account, Profile, Role.	*Đăng ký, Đăng nhập, Profile.*
Order Context	Trái tim của hệ thống. Tiếp nhận giỏ hàng, tính toán giá, quản lý trạng thái đơn hàng (từ lúc đặt đến lúc hoàn tất), xử lý thanh toán (gọi Gateway).	Order, OrderItem, Payment, ShoppingCart.	*Giỏ hàng, Đặt đơn, Checkout, Trạng thái Đơn hàng (Created, Paid, Cancelled...).*
Restaurant & Catalog Context	Quản lý thông tin nhà hàng, danh mục món ăn, giá cả, thực đơn. Xác nhận tính sẵn sàng của món ăn khi có đơn.	Restaurant, Menu, MenuItem, Category.	*Quán ăn, Món ăn, Menu, Còn hàng/Hết hàng.*
Kitchen/Merchant Order Context *(Mở rộng từ chức năng 2)*	Quản lý quy trình chuẩn bị món tại bếp sau khi đơn được xác nhận.	KitchenOrder (Tách biệt với Order tổng).	*Nhận đơn bếp, Đang chuẩn bị, Đã xong.*

Bounded Context	Trách nhiệm chính (Responsibility)	Các Entity/Aggregate chính	"Ngôn ngữ chung" (Ubiquitous Language) nổi bật
Delivery Context	Quản lý đội ngũ tài xế, gán đơn cho tài xế, theo dõi vị trí real-time, trạng thái giao hàng.	DeliveryJob, Driver, TrackingRoute.	*Tài xế, Nhận cuốc, Đang giao, Đã giao, Vị trí.*

3.Mô tả giao tiếp Sync vs Async giữa services.

Để hệ thống hoạt động mượt mà và chịu tải tốt, chúng ta cần kết hợp cả 2 hình thức:

3.1. Đồng bộ (Synchronous - Request/Response)

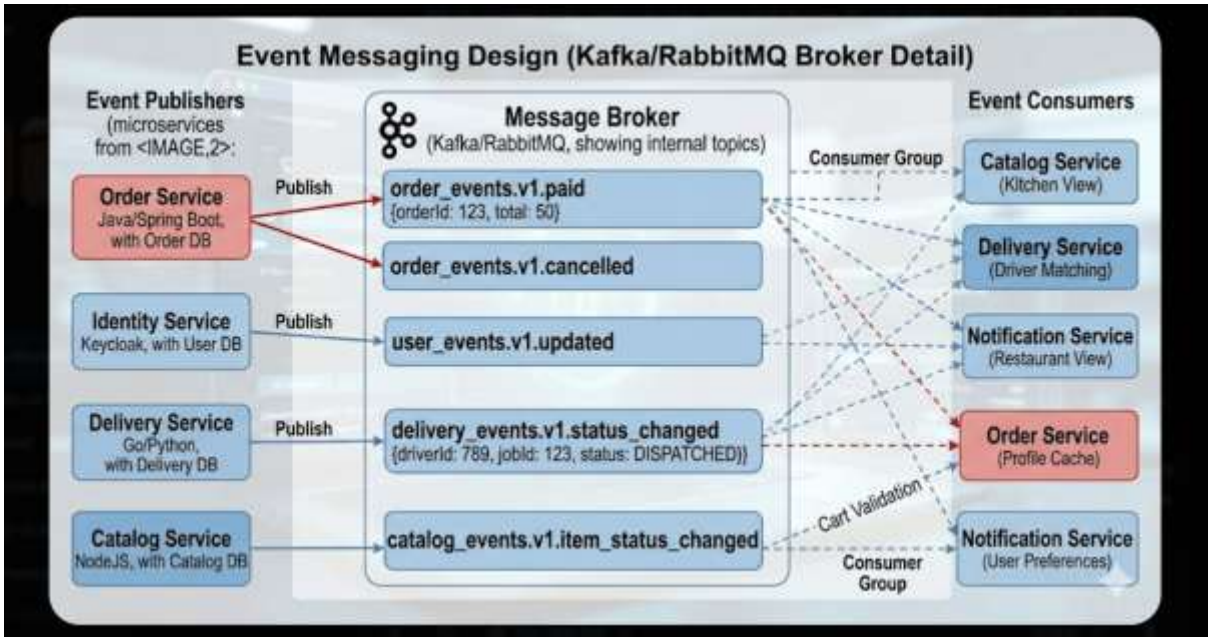
- Sử dụng khi: Cần kết quả ngay lập tức để phản hồi cho User, hoặc dữ liệu cần tính nhất quán tức thì (Strong Consistency).
- Công nghệ: HTTP/REST (phổ biến, dễ dùng), gRPC (hiệu năng cao, latency thấp, dùng nội bộ giữa các service).
- Ví dụ trong hệ thống:
    1. User Login: Khách hàng nhập pass -> API Gateway -> Identity Service (check DB) -> Trả về JWT. (Cần nhanh để login).
    2. Đặt hàng (Place Order) - Check Stock: Khi User ấn "Đặt đơn", Order Service phải gọi ngay (Sync) sang Catalog Service để xác nhận xem món ăn đó còn hàng không và giá có đúng không trước khi tạo đơn. Nếu Catalog Service die, việc đặt hàng phải failure ngay.

3.2. Bất đồng bộ (Asynchronous - Event-driven)

- Sử dụng khi: Hành động tốn thời gian, không cần phản hồi ngay cho User (Fire-and-forget), hoặc chấp nhận tính nhất quán sau (Eventual Consistency). Giúp tăng khả năng chịu tải (Scalability) và tính sẵn sàng (Availability).
- Công nghệ: Message Broker (Kafka, RabbitMQ).

- Ví dụ trong hệ thống (Rất quan trọng):
    1. Thanh toán thành công: Order Service nhận callback từ Payment Gateway -> Update DB -> Publish event OrderPaid lên Message Broker. Nó không cần đợi Notification Service gửi xong Email mới báo cho User.
    2. Chuẩn bị bếp: Bếp (Kitchen Service) listen event OrderPaid -> In đơn ở bếp.
    3. Tìm tài xế: Delivery Service listen event OrderPaid -> Bắt đầu thuật toán gán đơn cho tài xế gần nhất.

4. Tạo event messaging design (Kafka/RabbitMQ, chỉ cần mô hình).



Giải thích chi tiết mô hình Event:

Sơ đồ trên trình bày chi tiết cách Message Broker hoạt động như một hệ thống Pub/Sub cho 3 chức năng chính:

1. **Event Publishers (Bên trái):** Đây là các Microservices mà chúng ta đã tách ra. Khi có một sự kiện nghiệp vụ quan trọng xảy ra (ví dụ: OrderPaid, UserUpdated), chúng sẽ **Publish** (bắn) sự kiện đó vào một **Topic** cụ thể trong Broker.

    - *Ví dụ:* Khi Đơn hàng được thanh toán, Order Service bắn một message vào Topic order_events.v1.paid.
2. **Topics (Trong Broker):** Đây là các hàng đợi hoặc "kênh" dữ liệu được phân loại rõ ràng theo ngữ cảnh. Mỗi Topic chứa các message có cấu trúc tương tự nhau (Schemas).
    - *Ví dụ:* order_events.v1.paid chứa thông tin về đơn hàng đã trả tiền (như {orderId: 123, total: 50}).
3. **Event Consumers (Bên phải):** Đây là các services **đăng ký (Consume)** để nhận message từ các Topics mà chúng quan tâm. Một Topic có thể được nhiều service cùng Consume.
    - *Ví dụ:* Topic order_events.v1.paid được Consume song song bởi:
        - Catalog Service (để báo cho bếp/nhà hàng).
        - Delivery Service (để bắt đầu gán tài xế).
        - Notification Service (để gửi thông báo cho khách hàng).

Sơ đồ này làm rõ cách giao tiếp bất đồng bộ được tổ chức thông qua cấu trúc Topic, đảm bảo tính *Fire-and-Forget* và *Eventual Consistency* cho toàn hệ thống.